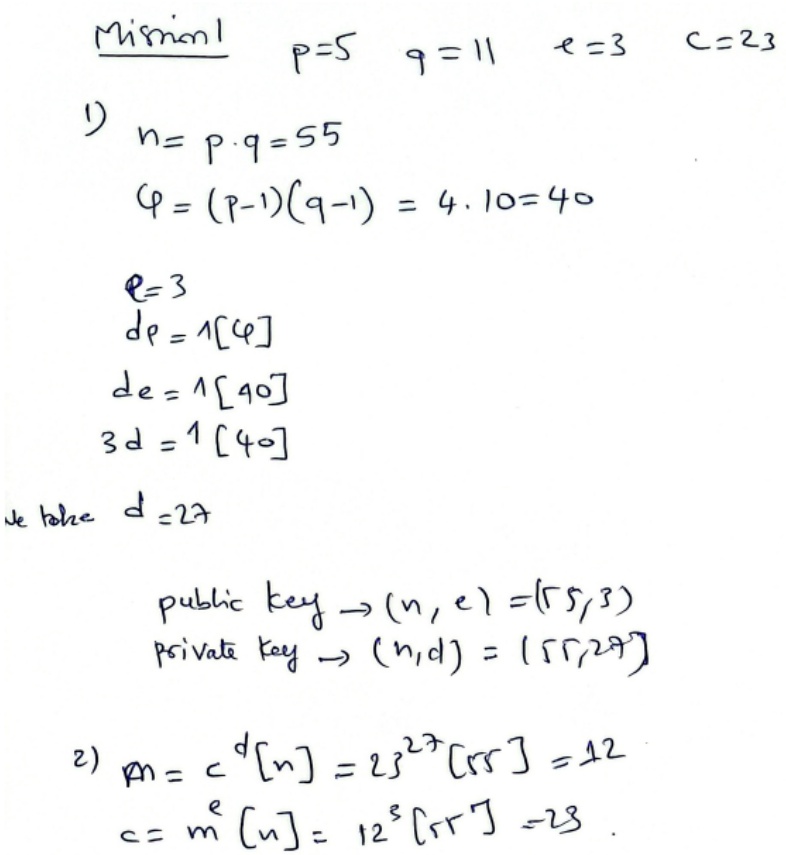
Lab : Mission 1 and 2

Houda TOUDALI

Information Security

Mission 1



Mission 2

Inspecting the encrypted file

I first checked the beginning of `classified.enc` with `xxd` and counted its bytes with `wc`.

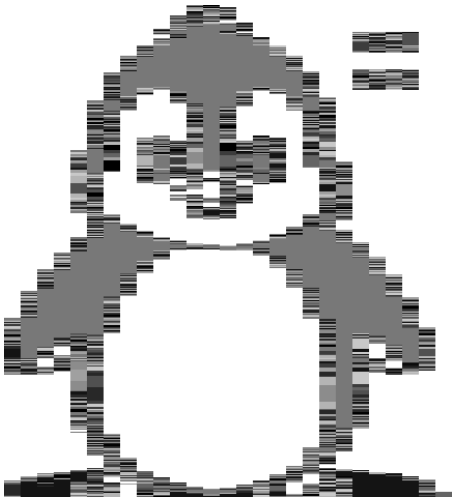
```
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 6$ xxd -l 64 classified.enc
00000000: 5361 6c74 656d 5f5f 575a b8cf 6019 ac39  Salted__WZ.`..9
00000010: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
00000020: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
00000030: 7775 1c07 8ddf 54b3 8a26 b7f6 f4a9 cef0  wu....T..&......
wwwto@houda:/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 6$ wc -c < classified.enc
205232
```

Checking the file header and size

The file starts with `Salted__` (8 bytes) and then the salt `575AB8CF6019AC39` (8 bytes). I skipped these first 16 bytes when analyzing the ciphertext. The file size is 205232 bytes, including the header and the AES padding.

Finding the ECB pattern

I divided the encrypted image data into blocks of 16 bytes. With Python, I counted identical blocks and gave the same gray value to matching blocks.



Pattern obtained from AES-ECB ciphertext

I could recognize the penguin and the text area, even without the password. The image is not decrypted here: ECB simply leaves repeated patterns visible because identical plaintext blocks give identical encrypted blocks.

Recovering the original image

I used the password **12** found in Mission 1 to decrypt the file with AES-256-ECB and PBKDF2.

```
wwwto@houda:~/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 6$ openssl enc -d -aes-256-ecb -pbkdf2 -p -in c
lassiied.enc -out recovered.raw
enter AES-36-ECB decryption password:
salt=575AB8CF6019AC39
key=103518D2C327E91924A746EA172211D62AC098CD2BA94E2270CC1D3612D6E877
```

OpenSSL decryption

The result was `recovered.raw`. I used Pillow to read it as a grayscale image of 432 × 475 pixels.



Recovered image

This time the penguin was clear, and I could read **YOU WIN** at the top.

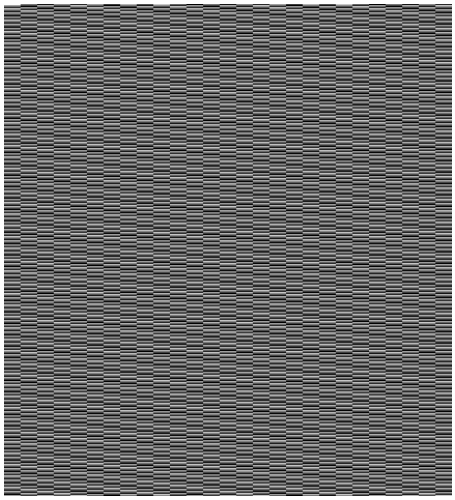
Encrypting again with CBC

I encrypted the recovered raw image again, this time using AES-256-CBC with PBKDF2.

```
wwwto@houda:~/mnt/c/Users/wwwto/Desktop/S3_CI2/Information Security/Labs/Lab 6$ openssl enc -aes-256-cbc -salt -pbkdf2 \
    -in recovered.raw \
    -out classified_cbc.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

Encrypting the image with AES-256-CBC

I then ran the same block analysis on the new encrypted file.



Pattern obtained from AES-CBC ciphertext

I could no longer recognize the penguin. In ECB, many blocks repeated (12086 extra copies), while in CBC there were no repeated ciphertext blocks in the image data I analyzed. CBC uses chaining, so the same plaintext block does not automatically produce the same ciphertext block.